Takum Arithmetic in a RISC-V Vector Extension: Functional Simulation and SpMV Evaluation

André Luís Alves Rezende, Gabriel Ramos Fontes
Isadora Maria Carvalho Lopes, João Pedro Gonçalves de Noronha Dutra
Department of Electronic Engineering, Federal University of Minas Gerais, Belo Horizonte, Brazil

***?abstractname?*—Low-precision formats add conversion, special-case, and decoding logic to vector instruction sets. Takum provides tapered precision, bounded dynamic range, and a shared 12-bit decoding window. This work proposes the experimental Zvtakum extension over RVV 1.0, with five custom-0 instructions for T8, T16, and T32. The arithmetic is implemented with integers and code-space round-to-nearest, ties-to-even. A functional RV64IM, Zicsr, RVV, and Zvtakum simulator is checked against QEMU for standard instructions and against an exact-rational Python model for Takum instructions and kernels. We evaluate sparse matrix–vector multiplication (SpMV) on eight Harwell–Boeing matrices using FP32 and Takum. The 96 kernel runs are bitwise equal to the reference model, and an analytic instruction-count model reproduces every run. FP8 and FP16 results are numerical baselines only: RVV 1.0 has no FP8, and Zvfh is not modeled.**

***Index Terms*—RISC-V, RVV, Takum arithmetic, Zvtakum, SpMV, functional simulator.**

I. Introduction

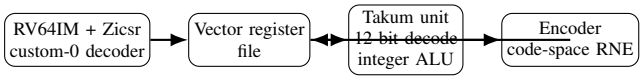
Vector instruction sets increasingly support low-precision formats such as bfloat16, FP16, and OCP FP8. Each format requires conversions and special-case rules, while RVV 1.0 does not define FP8 and FP16 requires a separate extension [1]–[3]. Takum is a tapered-precision format whose linear encoding uses a common 12-bit decoding window [4]. Hunhold's AVX10.2 study shows the potential of replacing several SIMD formats with a common Takum unit [5].

This paper makes the analogous study for RISC-V/RVV. Its contributions are an integer Takum unit, five custom-0 vector instructions, a functional simulator verified against both QEMU and an independent exact model, SpMV kernels with a common measurement perimeter, and an instruction-count model. The scope is functional: there is no hardware synthesis, timing, area, energy, logarithmic Takum variant, or vector Zvfh implementation.

II. Related Works

Low-precision vector arithmetic includes FP16, bfloat16, and OCP FP8 E4M3/E5M2. These formats have IEEE-like exponent and special-value behavior, whereas RVV 1.0 has no FP8 encoding and leaves FP16 to Zvfh. Takum and posits instead use tapered precision and a bounded representation [4], [6]. The present work evaluates both representation and vector instruction behavior, rather than only scalar conversion accuracy.

Hunhold's AVX10.2 work studies a Takum unit in an x86 SIMD ISA [5]; this work differs by targeting RISC-V/RVV, specifying custom-0 encodings, and measuring a functional simulator with explicit RVV tail and mask behavior. The simulator's arithmetic is integer-only and is checked against an exact-rational model, rather than relying on a C floating point intermediate. The VHDL codec provides related hardware evidence [7].



?figurename? 1. Functional path of a Zvtakum instruction: custom-0 decoding selects vector operands, the integer Takum unit performs the operation, and the code-space RNE encoder writes the destination.

The evaluation uses libtakum as an external reference for T8 [8], QEMU and LLVM for standard RISC-V behavior [9], [10], and the Harwell–Boeing matrices [11]. MuFoLAB is relevant infrastructure for larger studies [12], but the 1401-matrix experiment is outside this phase.

III. Methodology / Proposed Architecture

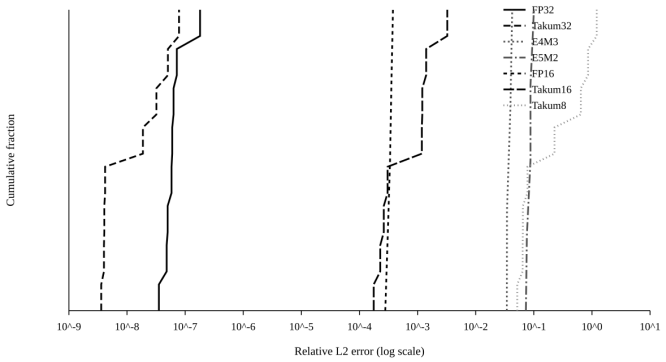
A. Architecture

The modeled processor contains RV64IM and Zicsr, a subset of RVV 1.0, and the Zvtakum unit. Figure 1 shows the path from custom-0 decoding through the vector register file and the Takum unit.

Zvtakum uses five custom-0 operations, checked against the opcode inventory [13]: `vtadd`, `vtsub`, `vtmul`, `vtdot`, and `vncvt`. They obey `vl`, `vstart`, mask `v0`, and RVV tail/mask agnostic policies. SEW is 8, 16, or 32 for ordinary operations; `vtdot` accumulates in 2SEW and `vncvt` narrows an aligned EMUL=2 group.

B. Takum encoding and arithmetic

The code contains sign S, direction D, three regime bits R, characteristic bits C, and fraction F. For widths below 12, decoding pads the code with phantom zero bits. The simulator represents a finite value by a normalized integer significand, an exponent, and a sticky bit. Encoding constructs the infinite code stream, retains an $n+1$-bit prefix, and uses the extra bit as the halfway bit; the sticky bit and the parity of the retained code implement code-space RNE. NaR propagates, zero has its specified neutral/absorbing behavior, and finite overflow saturates because Takum has no infinity.



?figurename? 2. Relative-error CDF against binary128. The versioned generator uses only existing result values and does not modify the CSVs.

C. SpMV and verification

The CSR SpMV kernels process rows in VLMAX-sized strips and use indexed vector loads. FP32 uses vector multiply and reduction; Takum uses `vtdot` and `vncvt`. The reference is binary128 arithmetic with separate multiply/add rounding. FP8 (E4M3/E5M2) and FP16 are evaluated only by the numeric baseline script: they have no simulator instruction count because FP8 is absent from RVV 1.0 and Zvfh is deliberately not modeled.

IV. Results and Discussion

The C++ Takum tests include complete T8 operation tables, sampled T16/T32/T64 operations, narrowing, dot products, and special values. Standard RVV behavior is differentially tested against QEMU when the required tools are available. The 96 SpMV executions are bitwise equal to the Python model.

With equal-width accumulators, Takum32 has a median relative error 2.01 times lower than FP32 in the tested matrices. The instruction model is $N_f = r_f R + c_f S + K$: `vtdot` replaces a multiply and reduction per strip, while `vncvt` adds one operation per row. Thus the Takum32 kernel is never more expensive than FP32 in the tested 24 matrix–VLEN pairs. These are paired results over eight matrices, not a claim about the full SuiteSparse collection.

The numeric baselines show the required limitation clearly. Takum8 and Takum16 do not overflow in the tested matrices, while E4M3, E5M2, and FP16 can produce non-finite rows. Among finite pairs, no low-precision format dominates every matrix. These FP8/FP16 observations concern relative error and overflow only; they do not include instruction counts or a vector Zvfh execution.

V. Conclusion

Zvtakum demonstrates a verified Takum arithmetic unit and vector instruction set for RVV. The integer implementation, simulator, exact model, and SpMV measurement perimeter agree bit for bit in the available tests. Takum32 combines lower error in the paired equal-accumulator comparison with an instruction count no greater than FP32 in this experiment. FP8/FP16 remain numeric baselines until a future phase models the corresponding hardware path. Future work includes Zvfh widening, larger matrix collections, and hardware cost evaluation.

?refname?

[1] RISC-V International, "The RISC-V vector extension, version 1.0," https://github.com/riscv/riscv-v-spec, 2021.

[2] P. Micikevicius, S. Oberman, P. Dubey *et al.*, "OCP 8-bit floating point specification (OFP8)," Open Compute Project, 2023.

[3] S. Wang and P. Kanwar, "BFloat16: The secret to high performance on cloud TPUs," 2019, google Cloud Blog.

[4] L. Hunhold, "Beating posits at their own game: Takum arithmetic," in *Next Generation Arithmetic (CoNGA 2024), LNCS vol. 14666*. Springer Nature Switzerland, 2024, arXiv:2404.18603.

[5] ——, "Streamlining SIMD ISA extensions with takum arithmetic: A case study on intel AVX10.2," in *2025 14th International Conference on Modern Circuits and Systems Technologies (MOCAST)*, 2025, pp. 1–6, arXiv:2503.14067.

[6] J. L. Gustafson and I. Yonemoto, "Beating floating point at its own game: Posit arithmetic," *Supercomputing Frontiers and Innovations*, vol. 4, no. 2, pp. 71–86, 2017.

[7] L. Hunhold, "Design and implementation of a takum arithmetic hardware codec in VHDL," 2024, arXiv:2408.10594.

[8] ——, "libtakum: Takum arithmetic C99 reference implementation, v2.0.0," https://github.com/takum-arithmetic/libtakum, 2025.

[9] "QEMU: emulador de processador e máquina, modo usuário," https://www.qemu.org, versão 10.2.2, CPU rv64 com extensão vetorial RVV 1.0.

[10] "LLVM mc: montador e desmontador para risc-v," https://llvm.org, versão 23.1.3.

[11] I. S. Duff, R. G. Grimes, and J. G. Lewis, "Sparse matrix test problems," *ACM Transactions on Mathematical Software*, vol. 15, no. 1, pp. 1–14, 1989.

[12] L. Hunhold and J. Quinlan, "MuFoLAB – multi-format linear algebra benchmarks," 2025.

[13] "riscv-opcodes: inventário de codificações risc-v," https://github.com/riscv/riscv-opcodes.

?tablename? I

RELATIVE L2 SPMV ERROR AT 32 BITS (VLEN = 256), BY MATRIX. FP32 IS THE VECTOR KERNEL (32-BIT ACCUMULATOR); CONTROLS USE THE NUMERICAL MODEL WITH THE INDICATED ACCUMULATOR.

Matrix	nnz	FP32	Takum32	FP32 (64-bit acc.)	Takum32 (32-bit acc.)
bcsstk01	400	7.20×10^{-8}	5.06×10^{-8}	3.57×10^{-8}	6.22×10^{-8}
bcsstk02	4356	1.81×10^{-7}	1.88×10^{-8}	4.63×10^{-8}	8.32×10^{-8}
jpwh_991	6027	6.32×10^{-8}	4.19×10^{-9}	3.47×10^{-8}	9.40×10^{-9}
pores_1	180	3.54×10^{-8}	7.87×10^{-8}	3.26×10^{-8}	5.44×10^{-8}
sherman1	3750	5.82×10^{-8}	4.02×10^{-9}	3.65×10^{-8}	4.93×10^{-9}
well1033	4732	5.99×10^{-8}	4.07×10^{-9}	3.72×10^{-8}	6.45×10^{-9}
west0067	294	4.82×10^{-8}	3.59×10^{-9}	3.27×10^{-8}	4.84×10^{-9}
west0132	413	5.01×10^{-8}	3.21×10^{-8}	4.00×10^{-8}	4.94×10^{-8}
Mediana		5.91×10^{-8}	1.15×10^{-8}	3.61×10^{-8}	2.94×10^{-8}

?tablename? II

RELATIVE L2 ERROR AT 8 AND 16 BITS, BY MATRIX (VLEN = 256). NF(k) DENOTES k NON-FINITE ROWS (OVERFLOW). ACCUMULATORS: E4M3, E5M2 AND TAKUM8 AT 16 BITS; FP16 AND TAKUM16 AT 32 BITS.

Matrix	E4M3	E5M2	FP16	Takum16	Takum8
bcsstk01	NF(48)	NF(48)	NF(48)	3.24×10^{-3}	6.44×10^{-1}
bcsstk02	NF(66)	7.31×10^{-2}	3.76×10^{-4}	1.20×10^{-3}	2.27×10^{-1}
jpwh_991	3.45×10^{-2}	7.58×10^{-2}	2.77×10^{-4}	3.04×10^{-4}	7.77×10^{-2}
pores_1	NF(30)	NF(15)	NF(15)	1.18×10^{-3}	8.58×10^{-1}
sherman1	3.95×10^{-2}	1.00×10^{-1}	3.06×10^{-4}	2.28×10^{-4}	6.40×10^{-2}
well1033	4.24×10^{-2}	8.75×10^{-2}	3.54×10^{-4}	2.62×10^{-4}	6.47×10^{-2}
west0067	3.43×10^{-2}	8.74×10^{-2}	3.36×10^{-4}	1.76×10^{-4}	5.18×10^{-2}
west0132	NF(17)	NF(3)	NF(3)	1.40×10^{-3}	1.21×10^{0}

?tablename? III

INSTRUCTIONS, ZVTAKUM INSTRUCTIONS, CSR WRITES, AND ACTIVE ELEMENTS PER NONZERO, SUMMED OVER THE EIGHT MATRICES. ACTIVE ELEMENTS COUNT EACH ITEM PROCESSED BY VECTOR DATA INSTRUCTIONS (LOAD, GATHER, PRODUCT, REDUCTION, STORE).

Format	VLEN	Instr./nnz	Zvtakum/nnz	CSR writes/nnz	Active elem./nnz
FP32	128	6.140	0.000	1.328	5.334
FP32	256	4.587	0.000	1.069	5.334
FP32	512	4.343	0.000	1.029	5.334
Takum32	128	5.977	0.497	1.328	4.501
Takum32	256	4.553	0.368	1.069	4.501
Takum32	512	4.329	0.347	1.029	4.501
Takum16	128	4.954	0.368	1.470	4.501
Takum16	256	4.690	0.347	1.389	4.501
Takum16	512	4.605	0.341	1.363	4.501
Takum8	128	4.510	0.347	1.389	4.501
Takum8	256	4.431	0.341	1.363	4.501
Takum8	512	4.392	0.337	1.350	4.501

?tablename? IV

PARÂMETROS DO MODELO DE CONTAGEM $N = rR + cS + K$, EM QUE R É O NÚMERO DE LINHAS E S O NÚMERO DE FAIXAS DAS LINHAS NÃO VAZIAS. A ÚLTIMA COLUNA COMPARA O MODELO COM AS EXECUÇÕES REAIS.

Format	r	c	K	Synthetic	Exact runs
FP32	13	12	21	3/3	24/24
Takum8	14	12	21	3/3	24/24
Takum16	14	13	21	3/3	24/24
Takum32	14	11	21	3/3	24/24